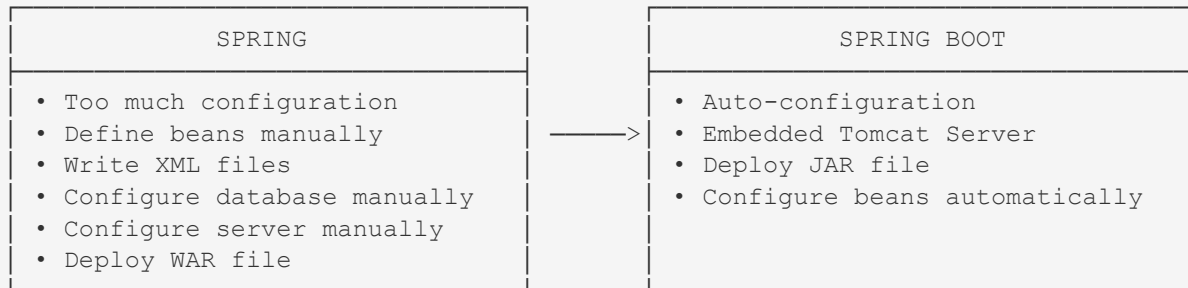


Spring Boot & JPA

What is Spring Boot?

Spring Boot is an open-source Java-based framework built on top of the Spring Framework that simplifies the development, configuration, and deployment of standalone, production-ready applications by providing auto-configuration, embedded servers, and opinionated defaults.

What is Spring and why Spring Boot has come?



- In **Spring**, you need to add everything manually, for example:
- Spring MVC
- Jackson
- Tomcat
- In **Spring Boot**, you just add `spring-boot-starter-web` and everything comes together automatically.

What is JAR and what is WAR?

JAR (Java Archive)

- Runs using JVM.
- Used for normal Java applications.
- Run command:

```
java -jar app.jar
```

WAR (Web Archive)

- Used for web applications.
- Must be deployed on an external Server (Tomcat, JBoss, WebLogic, etc.).
- **Deployment Flow:**
WAR → TOMCAT → RUN APP

Traditional Spring (Why WAR was used?)

- In old Spring applications, there was no embedded server.
- The application depended on an external server (like Tomcat) to run (i.e., the application had to be deployed into the external server).
- In Spring Boot, we have an **embedded server**, so we can deploy as a **JAR** file.

IoC (Inversion of Control)

1. **IoC** means Spring controls object creation and dependency management, not the developer.
2. When you run a Spring Boot application using `SpringApplication.run(...)`, it creates an `ApplicationContext`. This context is the IoC container that manages all beans.

Different IoC Containers and How to Create Them

① *BeanFactory*

- The simplest IoC container in Spring.
- Provides basic dependency injection features.
- Creates beans lazily (on-demand using `getBean()`).

How to create:

```
public static void main(String[] args) {  
    // Load beans.xml from classpath  
    BeanFactory factory = new XmlBeanFactory(new ClassPathResource("beans.xml"));  
  
    // Get bean from container  
    Car car = (Car) factory.getBean("car");  
    car.drive();  
}
```

- `ClassPathResource("beans.xml")` → Points to the XML file in the classpath.
- `XmlBeanFactory` → Reads bean definitions from that XML file.
- `getBean("car")` → Instantiates the `car` object only when requested (lazy initialization).

② *ApplicationContext*

- A superset of `BeanFactory` with additional enterprise-grade features:

1. Event handling
 2. Internationalization (i18n)
 3. AOP (Aspect-Oriented Programming) integration
 4. Eager bean creation (beans are created on startup)
-

Types of ApplicationContext

Ⓐ *ClassPathXmlApplicationContext*

Loads bean definitions from an XML file located in the classpath.

```
public class TestApplicationContext {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");

        Car car = context.getBean(Car.class);
        car.drive();
    }
}
```

Ⓑ *FileSystemXmlApplicationContext*

Loads bean definitions from an XML file located in a specific directory on the filesystem.

```
ApplicationContext context = new
FileSystemXmlApplicationContext("c:/spring/beans.xml");

Car car = context.getBean(Car.class);
```

Ⓒ *AnnotationConfigApplicationContext*

Loads beans from Java-based configuration classes (annotated with `@Configuration`).

Example:

```
@Configuration
@ComponentScan("com.example")
public class AppConfig { }

public class TestAnnotationConfig {
    public static void main(String[] args) {
        ApplicationContext context = new
        AnnotationConfigApplicationContext(AppConfig.class);

        Car car = context.getBean(Car.class);
        car.drive();
    }
}
```

③ *Spring Boot IoC Container*

- Spring Boot always uses `ApplicationContext` (typically `AnnotationConfigApplicationContext` or `GenericWebApplicationContext` under the hood).
- It is created automatically when you run the application.

How it is created:

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(MyApp.class, args);

        Car car = context.getBean(Car.class);
        car.drive();
    }
}
```

No need for XML files or manual container setup.

Dependency Injection (DI)

- Dependency Injection is a design pattern used in Spring to implement Inversion of Control (IoC).
- Instead of a class creating its own dependencies using `new`, the IoC container provides (injects) those dependencies.

① Constructor Injection

```
@Component
public class Car {
    private final Engine engine;

    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

② Setter Injection

- Dependencies are injected via setter methods.

```
@Component
public class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

③ Field Injection

- Dependencies are injected directly into fields using `@Autowired`.

```
@Component
public class Car {
    @Autowired
    private Engine engine;
}
```

Key Spring Annotations

- `@Autowired`: Used for Dependency Injection. It tells the IoC container: *"Find a bean of type Engine and inject it here."*
- `@Component`: It tells the IoC container: *"Please create an object of this class, manage its lifecycle, and make it available for dependency injection."*

Stereotype Annotations (Specializations of `@Component`):

1. `@Controller` (Presentation layer - Web MVC)
2. `@RestController` (RESTful Web Services - `@Controller` + `@ResponseBody`)
3. `@Service` (Service / Business logic layer)
4. `@Repository` (Data Access / DAO layer)
5. `@Configuration` (Defines Java-based bean configurations)
6. `@ControllerAdvice` / `@RestControllerAdvice` (Global exception handling and response formatting)

Why do we use these specializations?

They make the code more readable, self-documenting, and support layered architecture:

CONTROLLER → SERVICE → REPOSITORY

① `@Controller`

- A specialization of `@Component` used in the presentation layer of a Spring MVC application.
- It marks a web controller that handles HTTP requests and returns views (HTML pages).

Example:

```
@Controller
@RequestMapping("/user")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id, Model model) {
```

```

        User user = userService.findById(id);
        model.addAttribute("user", user);
        return "userProfile"; // Returns HTML view name
    }
}

```

② `@RestController`

- Used in REST APIs to return data (JSON/XML) directly.
- Combines `@Controller` + `@ResponseBody`. No need to add `@ResponseBody` manually on every method.

Example:

```

@RestController
@RequestMapping("/api/users")
public class UserApiController {
    private final UserService service;

    public UserApiController(UserService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return service.findById(id); // Returns Java Object -> Automatically
        converted to JSON
    }
}

```

If you want to use `@Controller` to return JSON/XML data instead of a template view, you must add `@ResponseBody` to the method or class:

```

@Controller
public class ApiController {
    @GetMapping("/api/message")
    @ResponseBody
    public String message() {
        return "Hello World";
    }
}

```

CONCLUSION:

`@Controller` → @component + specialization

`@RestController` → `@ResponseBody` + `@Controller`

Difference between `@RequestMapping` vs. `@GetMapping` / `@PostMapping` etc.

- `@RequestMapping` is the general-purpose annotation for mapping HTTP requests to controller methods.
- Specialized shortcuts like `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, and `@PatchMapping` are specific mapping shortcuts.

General Example:

```
@RequestMapping(value = "/users", method = RequestMethod.GET)
public List<User> getUsers() {
    return userService.findAll();
}
```

- `@RequestMapping` is often used at the class level to set a base path, or for complex mappings containing custom headers, params, and multiple HTTP methods.

Key Parameters of `@RequestMapping`

1. `value` / `path``: Defines the URL pattern(s) for the request.

- Example: `@RequestMapping("/users")`

2. `method``: Restricts the mapping to specific HTTP methods (GET, POST, PUT, DELETE, PATCH).

- Example: `@RequestMapping(value = "/users", method = RequestMethod.GET)`

3. `consumes``: Specifies the media type the method can consume (Input).

- Example: `@RequestMapping(value = "/users", consumes = "application/json")`

4. `produces``: Specifies the media type the method can produce (Output).

- Example: `@RequestMapping(value = "/users", produces = "application/json")`

5. `headers``: Matches requests based on the presence or value of HTTP headers.

Scenarios for `headers` attribute`:

Ⓐ Presence only

Requires that a header exists, regardless of its value.

```
@RequestMapping(value = "/secure", headers = "X-Auth-Token")
```

Matches if `X-Auth-Token` is present in the request header (value can be anything, even empty).

ⓑ Specific Value

Requires that a header has a particular value.

```
@RequestMapping(value = "/users", headers = "X-API-VERSION=2")
```

Matches only if `X-API-VERSION` is present and equals `2`.

ⓒ Multiple headers together

```
@RequestMapping(value = "/report", headers = {"X-API-VERSION=1",  
"Accept=application/json"})
```

Matches only if both headers are present with those exact values.

ⓓ Negation (absence)

Requires that a header is **not** present in the request.

```
@RequestMapping(value = "/default", headers = "!X-API-VERSION")
```

Matches only if `X-API-VERSION` is missing from headers.

ⓔ Combination of presence and value

```
@RequestMapping(value = "/special", headers = {"X-Auth-Token", "Client-  
Type=Mobile"})
```

Matches if `X-Auth-Token` exists and `Client-Type` equals `Mobile`.

Real-world Scenario: API Versioning

Headers can be used for versioning. If API Version 1 returns `{ "name", "age" }` and Version 2 returns `{ "name", "age", "salary" }`, you do not need to rewrite the URL. You can map different methods to the same URL path but differentiate them using headers.

Handling Controller Inputs

① `@PathVariable`

Extracts values directly from the URI path.

- **Request:** `GET /students/10`

```
@GetMapping("/students/{id}")  
public String getStudent(@PathVariable Long id) {  
    return "Student ID: " + id;  
}
```


② `@RequestParam`

Used for query parameters in the URL.

- **Request:** `GET /students?name=Vineeth`

```
@GetMapping("/students")
public String getStudentByName(@RequestParam String name) {
    return "Student Name: " + name;
}
```

You can also make a query parameter optional:

```
@RequestParam(required = false) String name
```

③ `@RequestBody`

Used to bind the HTTP request body to a Java object.

- **Request JSON:**

```
{
  "name": "Vineeth",
  "age": 22
}
```

- **Controller:**

```
@PostMapping("/students")
public Student createStudent(@RequestBody Student student) {
    return student;
}
```

Spring automatically converts JSON → Java object using the Jackson library.

④ `@RequestHeader`

Used to extract header values from the incoming HTTP request.

```
@GetMapping("/header")
public String getHeader(@RequestHeader("Authorization") String token) {
    return token;
}
```

⑤ `@RequestParam` (File Upload)

Used for multi-part request contents (such as files).

```
@PostMapping("/upload")
public String uploadFile(@RequestPart MultipartFile file) {
    return file.getOriginalFilename();
}
```

`@Service` Annotation

- Tells Spring: `"This class is a service component. Create an object of this class and manage it as a Spring Bean."`
- Under the hood, `@Service` is meta-annotated with:
- `@Target(ElementType.TYPE)`
- `@Component`
- `@Retention(RetentionPolicy.RUNTIME)`
- `@Documented`

`@Repository` Annotation

- Used to mark the data access layer (DAO) repository classes.
- It inherits the same stereotype configurations as `@Service` and `@Component` but adds automatic database exception translation.

Spring Data JPA (Java Persistence API)

Let's look at the history of how ORM/JPA/Hibernate was formed.

1. Before Hibernate and JPA — How Java Connected to Database

Initially, Java applications used **JDBC (Java Database Connectivity)**.

JDBC is a standard Java API used to:

1. Connect to a database.
2. Execute SQL queries.
3. Fetch results.

Example using JDBC:

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/test", "root", "Password"
);
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO Student(id, name) VALUES (?,?)"
);
```

Problems with JDBC:

1. **Too much boilerplate code:** Setting up connections, statements, try-catch blocks, and resource closing.
2. **Manual Object Mapping:** Iterating `ResultSet` and manually mapping columns to object fields.
3. **SQL Everywhere:** Business logic code is heavily mixed with database-specific SQL.
4. **Database Dependency:** Switching databases (e.g., MySQL & PostgreSQL) requires significant changes.

The core problem was that Java Objects and Relational Databases are structured differently. This mismatch is called **Object-Relational Impedance Mismatch**.

2. Hibernate was Introduced

To solve JDBC complexity, ORM (Object Relational Mapping) frameworks were created.

What ORM Means:

ORM automatically maps:

- **Java Class → Database Table**
- **Java Object → Database Row**
- **Java Field → Database Column**

Example:

- Class Name `Student` & Table `student`

Main Goal of Hibernate:

Hibernate automatically:

1. Converts Java objects → SQL statements.
2. Converts SQL result sets → Java objects.

Example:

```
studentRepository.save(student);
```

Hibernate internally generates and executes:

```
INSERT INTO Student(name) VALUES ('Vineeth');
```

3. Problems After Hibernate

Although Hibernate solved database interaction, other ORM frameworks emerged (e.g., TopLink, OpenJPA), each with different APIs.

Main Problem:

If a company switched its ORM framework, the entire codebase had to be rewritten. This created a **Vendor Lock-In** problem.

4. JPA Was Introduced

To solve Vendor Lock-In, Java introduced **JPA (Java Persistence API)**.

What is JPA?

JPA is a **specification/standard** for ORM in Java.

JPA is **not** a framework. It only defines:

- * Rules
- * Interfaces
- * Annotations
- * Standards

JPA tells vendors: *"Your ORM framework must behave like this."*

5. Relationship Between JPA and Hibernate

- Hibernate is an implementation of the JPA specification.
- JPA itself is just a set of rules and interfaces and does not contain execution logic.

Core Entity Mapping

① `@Entity`

Tells Spring/Hibernate: *"This Java class should be mapped to a database table."*

Example:

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
}
```

What Hibernate Creates Internally (Entity Metadata):

- Class name

- Table name
- Fields
- Columns
- Primary key

Why Metadata is Needed?

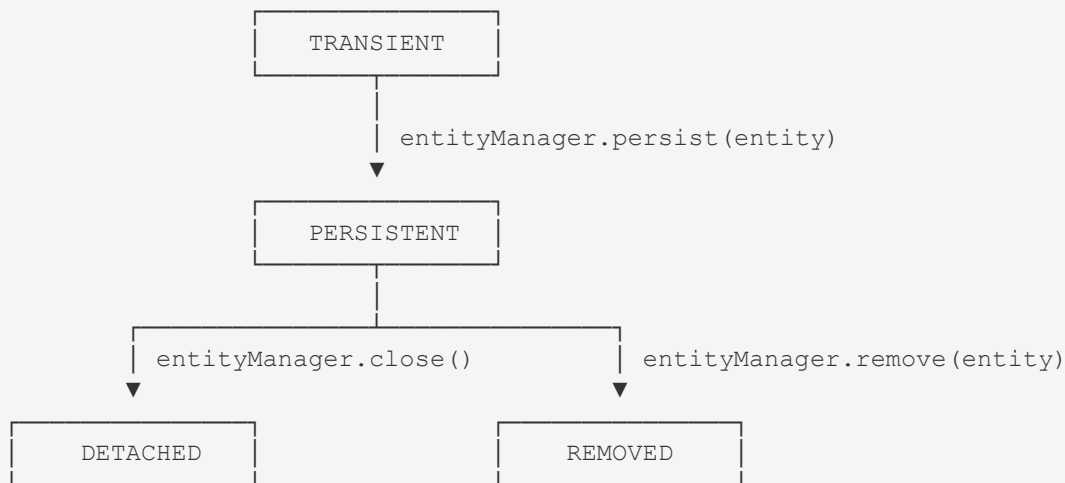
Hibernate uses this metadata for:

- SQL statement generation
- Object mapping
- Dirty checking (detecting changes in objects)
- Caching
- Lifecycle management

Custom Table Mapping:

```
@Entity
@Table(name = "students")
public class Student {
}
```

Entity Lifecycle States



1. Transient State:

- Object only exists in JVM memory.
- No database record and no Hibernate session connection.
- Hibernate is unaware of the object.

2. Persistent (Managed) State:

- Achieved when calling `entityManager.persist(student)`.
- The object is now managed by Hibernate and tracked for changes.

3. Detached State:

- Occurs if the Hibernate session/EntityManager is closed: `entityManager.close()`.
- The object still exists in JVM, but changes will not be synced to the database automatically.

4. Removed State:

- Achieved via `entityManager.remove(student)`.
- Marks the entity for deletion from the database.

② `@Id`

Marks the field representing the primary key of the database table.

③ `@GeneratedValue`

Defines the strategy for primary key generation.

- Example:

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

④ `@Column`

Used to customize column details.

Example:

```
@Column(
    name = "student_name",
    nullable = false,
    unique = true,
    length = 50,
    updatable = false
)
private String name;

@Column(
    precision = 10,
    scale = 2,
    columnDefinition = "DECIMAL(10,2) DEFAULT 0.0"
)
private BigDecimal salary;
```

⑤ *Data Types Mapping*

- `String` &rrar; `VARCHAR`
- `LocalDate` &rrar; `DATE` (date only)
- `LocalDateTime` &rrar; `TIMESTAMP` (date + time)
- `enum` (using `@Enumerated(EnumType.STRING)`) &rrar; Recommended
- `byte[]` / `Blob` &rrar; Binary Large Object
- `Clob` &rrar; Character Large Object
- `@Lob` &rrar; Enables large object storage (BLOB/CLOB)

Repository + CRUD Operations

Spring Data JPA repositories expose built-in CRUD operations:

1. `save()`
2. `saveAll()`
3. `findById()`
4. `existsById()`
5. `getReferenceById()`
6. `delete()`
7. `flush()`

SOLID Principles

① Single Responsibility Principle (SRP)

- A class should perform **only one** specific job.
- If a class has multiple responsibilities, changes in one responsibility may break or affect the other.
- **Rule:** One class = One Responsibility = One Reason to change.

Bad Design (Violates SRP):

```
class Employee {  
    public void calculateSalary() {  
        // Business logic  
    }  
  
    public void saveToDatabase() {  
        // DB operation  
    }  
  
    public void generateReport() {  
        // Reporting  
    }  
}
```

- **Problem:** This class has 3 responsibilities (Salary, Database, Report). Thus, it has 3 reasons to change.

Good Design (Follows SRP):

```
class Employee {  
    // Basic employee fields  
}  
  
class SalaryService {
```

```

    public void calculateSalary(Employee emp) {
        // Only salary logic
    }
}

class EmployeeRepository {
    public void save(Employee emp) {
        // Only database operations
    }
}

class ReportService {
    public void generateReport(Employee emp) {
        // Only report generation logic
    }
}

```

- Each class has only one responsibility.

② Open/Closed Principle (OCP)

- A class should be **open for extension**, but **closed for modification**.

Bad Design (Violates OCP):

```

class PaymentService {
    public void pay(String type) {
        if(type.equals("UPI")) {
            System.out.println("UPI payment");
        } else if (type.equals("CARD")) {
            System.out.println("Card payment");
        }
    }
}

```

- **Problem:** Every time a new payment method (e.g., NetBanking) is introduced, you must modify this `pay` method, risking breaking existing code.

Good Design (Follows OCP):

```

interface Payment {
    void pay();
}

class UpiPayment implements Payment {
    public void pay() {
        System.out.println("UPI Payment");
    }
}

class CardPayment implements Payment {
    public void pay() {
        System.out.println("Card Payment");
    }
}

```



```
}  
}
```

- **Advantage:** We can easily add new payment types (e.g., `NetBankingPayment`) by creating a new class without modifying the existing interfaces/classes.

③ Liskov Substitution Principle (LSP)

- Child classes should be able to replace parent classes without breaking the application.
- The child class must behave exactly like the parent class for inherited behaviors.

Bad Design (Violates LSP):

```
class Bird {  
    void fly() {  
        System.out.println("Flying");  
    }  
}  
  
class Ostrich extends Bird {  
    void fly() {  
        throw new RuntimeException("Ostrich can't fly");  
    }  
}
```

- **Problem:** `Ostrich` breaks the expected behavior of the parent class `Bird` by throwing an exception on `fly()`.

Good Design (Follows LSP):

```
class Bird {  
    // General bird properties  
}  
  
class FlyingBird extends Bird {  
    void fly() {  
        System.out.println("Flying");  
    }  
}  
  
class Sparrow extends FlyingBird {  
    // Sparrow behaves like FlyingBird  
}  
  
class Ostrich extends Bird {  
    // Ostrich only inherits general Bird behaviors, not FlyingBird  
}
```

- No forced behaviors or broken substitutions.

④ Interface Segregation Principle (ISP)

- A class should not be forced to implement methods it does not use.

Bad Design (Violates ISP):

```
interface Worker {
    void work();
    void eat();
}

class Robot implements Worker {
    public void work() {
        System.out.println("Working");
    }

    public void eat() {
        // Robots do not eat! Empty implementation.
    }
}
```

- **Problem:** `Robot` is forced to implement `eat()`, which it doesn't need.

Good Design (Follows ISP):

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class Human implements Workable, Eatable {
    public void work() {
        // Human works
    }

    public void eat() {
        // Human eats
    }
}

class Robot implements Workable {
    public void work() {
        // Robot works
    }
}
```

- Clean separation of interfaces.

⑤ Dependency Inversion Principle (DIP)

- Depend on abstractions, not on concrete implementations.

Bad Design (Violates DIP):

```
class EmployeeService {
    private MySQLDatabase db = new MySQLDatabase();

    void save() {
        db.save();
    }
}
```

- **Problems:**

1. `EmployeeService` is tightly coupled to the concrete class `MySQLDatabase`.
2. `EmployeeService` manages the database lifecycle itself (`new MySQLDatabase()`).
3. It is hard to switch databases or write unit tests.

Good Design (Follows DIP):

- High-level classes should not depend on low-level classes. Both should depend on abstractions (interfaces).

```
interface Database {
    void save();
}

class MySQLDatabase implements Database {
    public void save() {
        System.out.println("Saving to MySQL");
    }
}

class OracleDatabase implements Database {
    public void save() {
        System.out.println("Saving to Oracle");
    }
}

class EmployeeService {
    private Database db;

    // Database dependency injected from outside
    public EmployeeService(Database db) {
        this.db = db;
    }

    void save() {
        db.save();
    }
}
```

What DIP Achieves

Before DIP:

EmployeeService → MySQLDatabase

After DIP:

EmployeeService → Database (abstraction)

MySQLDatabase → Database

OracleDatabase → Database

`EmployeeService` does not know anything about the concrete classes. It only knows the `Database` abstraction.

New Question:

If `Database db` is an interface, who will instantiate the concrete `MySQLDatabase` or `OracleDatabase` object to pass it into the constructor?

- **Answer: Dependency Injection (DI).**
- Instead of the class instantiating its own dependencies, we inject them from the outside.
- In Spring, this injection is automatically handled by the **IoC Container**.

`@Primary` and `@Qualifier`

Suppose we have:

```
interface Database { }

@Component
class MySQLDatabase implements Database { }

@Component
class OracleDatabase implements Database { }
```

And inject it like this:

```
@Service
class EmployeeService {
    @Autowired
    private Database database;
}
```

Spring Default

Spring scans and finds two beans of type `Database` (`MySQLDatabase` and `OracleDatabase`). It gets confused about which one to inject. This results in a `NoUniqueBeanDefinitionException`.

① `@Primary`

Used to tell Spring: `"If multiple beans of the same type exist, pick this one by default."`

```
@Component
@Primary
class MySQLDatabase implements Database {
}
```

Now, `MySQLDatabase` is injected as the default.

② `@Qualifier`

Used when you want to explicitly name and pick a specific bean to inject.

Step 1: Assign names to beans

```
@Component("mySqlDb")
class MySQLDatabase implements Database { }

@Component("oracleDb")
class OracleDatabase implements Database { }
```

Step 2: Inject with choice

```
@Service
class EmployeeService {
    private final Database database;

    @Autowired
    public EmployeeService(@Qualifier("oracleDb") Database database) {
        this.database = database;
    }
}
```

Now, `OracleDatabase` is injected explicitly.

`@Configuration` + `@Bean`

- A way to manually declare Spring beans using Java configuration.
 - **Why do we need it?**
 1. For third-party or built-in classes where you cannot add `@Component` annotation directly to the source code.
 2. When custom parameters are needed during object instantiation.
-

Example:

```
@Configuration
class AppConfig {
```

```
@Bean
public Database database() {
    return new MySQLDatabase();
}
}
```

Using `@Profile`

- `@Profile` is used to switch bean configurations based on different environments (e.g., `dev`, `prod`, `test`).
- **Example:**

```
@Configuration
class DataSourceConfig {
    @Bean
    @Profile("dev")
    public Database devDatabase() {
        return new MySQLDatabase(); // Local dev database
    }

    @Bean
    @Profile("prod")
    public Database prodDatabase() {
        return new OracleDatabase(); // Production database
    }
}
```

To activate a profile, configure it in `application.properties`:

```
spring.profiles.active=dev
```

When running the application JAR file, you can set the active profile via the command line:

```
java -jar app.jar --spring.profiles.active=prod
```

Typically, different configurations are stored in separate files:

- `application-dev.properties`
- `application-test.properties`

Which of the 3 Dependency Injection types is best?

- **Answer:** Constructor Injection.

Let's analyze why:

① Field Injection

Suppose we have:

```
@Component
class Engine {
    public void run() {
        System.out.println("Engine started");
    }
}
```

`Car.java`

```
class Car {
    Engine engine;

    public void start() {
        engine.run();
        System.out.println("Car started");
    }
}
```

`Main.java`

```
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
    }
}
```

Output:

```
Exception in thread "main" java.lang.NullPointerException
```

- **Explanation:** Since we never instantiated `Engine`, `engine` is `null`. Calling `engine.run()` inside `car.start()` triggers a `NullPointerException`.

How to Solve Manually:

We must manually set the Engine dependency:

```
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.engine = new Engine();
    }
}
```

```
        car.start();
    }
}
```

Output:

```
Engine started
Car started
```

Problem with Field Injection:

CAR.JAVA

```
@Component
class Car {
    @Autowired
    private Engine engine;

    public Car() {
        engine.run();
    }
}
```

ENGINE.JAVA

```
@Component
class Engine {
    public void run() {
        System.out.println("Engine..");
    }
}
```

Constructor execution order:

1. Spring instantiates `Car` by invoking `Car car = new Car()`.
2. While executing the constructor, `engine` is still `null` because field injection occurs **after** construction.
3. Running `engine.run()` inside the constructor throws a `NullPointerException`.

② Setter Injection

Injected via setter methods instead of fields.

`Engine.java`

```
class Engine {
    public void run() {
        System.out.println("Engine Started");
    }
}
```


`Car.java`

```
class Car {  
    private Engine engine;  
  
    public void setEngine(Engine engine) {  
        this.engine = engine;  
    }  
  
    public void start() {  
        engine.run();  
        System.out.println("Car started");  
    }  
}
```

`Main.java`

```
public class Main {  
    public static void main(String[] args) {  
        Engine engine = new Engine();  
        Car car = new Car();  
        car.setEngine(engine);  
        car.start();  
    }  
}
```

Output:

```
Engine started  
Car started
```

- **Downside:** If you forget to call `car.setEngine(engine)` during instantiation, you will get a `NullPointerException`. Setter injection does not enforce dependency fulfillment at compile time.

③ *Constructor Injection*

`Engine.java`

```
class Engine {  
    public void run() {  
        System.out.println("Engine Started");  
    }  
}
```

`Car.java`

```
class Car {  
    private final Engine engine;  
  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
  
    public void start() {  
        engine.run();  
        System.out.println("Car Started");  
    }  
}
```

`Main.java`

```
public class Main {  
    public static void main(String[] args) {  
        Engine engine = new Engine();  
        Car car = new Car(engine);  
        car.start();  
    }  
}
```

Memory Behavior

1. `Engine engine = new Engine();` → Creates an Engine object in the heap.
2. `Car car = new Car(engine);` → Invokes the constructor and assigns the engine reference immediately.

Try this:

```
Car car = new Car(); // Compilation error!
```

- **Compiler Error:** `Constructor Car() is undefined.`
- **Advantage:** Java forces the developer to provide an `Engine` to construct a `Car`. You cannot create an incomplete or broken `Car` object.

Therefore, **Constructor Injection is the best option** (forces dependencies, enables final fields, avoids NullPointerExceptions).

Spring Boot Actuator

- A monitoring and management tool for running Spring Boot applications.
- Exposes internal application details through HTTP REST endpoints.

\$\$\text{Actuators} = \text{A tool exposing internal application details via HTTP endpoints}\$\$

WHY USE ACTUATORS?

Spring Boot runs silently in the background:

- Creates beans.
- Starts the embedded server.
- Connects to the database.

Actuators provide visibility to check:

- Is the application running?
- Is the database connected and working?
- What beans have been created?
- Which API routes are mapped?
- What is the memory and CPU usage?

Accessing Endpoints

By default, endpoints are exposed under `/actuator`:

- `http://localhost:8080/actuator`
- `http://localhost:8080/actuator/beans`

How to Add Actuator

Maven Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

By default, Spring Boot does not expose all endpoints due to security reasons. Only `/actuator/health` and `/actuator/info` are visible out of the box.

Exposing Endpoints:

You can configure which endpoints are exposed in `application.properties`:

```
# Exposes only the 'beans' endpoint
management.endpoints.web.exposure.include=beans

# Exposes multiple specific endpoints
management.endpoints.web.exposure.include=health,beans

# Exposes ALL endpoints (use with caution in prod)
management.endpoints.web.exposure.include=*
```

`/actuator/health` Statuses

Status	Description
UP	The application is healthy and working normally.
DOWN	The application or a dependency (e.g. Database) has failed.
OUT_OF_SERVICE	The application is in maintenance mode.
UNKNOWN	Status cannot be determined.

You can customize health status indicators by implementing the `HealthIndicator` interface.

Endpoint Output Example:

Accessing `http://localhost:8080/actuator` returns JSON links:

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    }
  }
}
```

- `templated: false` & rarr; Fixed static URL.
- `templated: true` & rarr; URL template that accepts path variables.

`@ComponentScan`

- Component scanning is the process where Spring automatically scans packages for classes annotated with stereotype annotations (`@Component`, `@Service`, `@Repository`, `@Controller`), instantiates them as beans, and registers them in the IoC Container.
- Package: `org.springframework.context.annotation.ComponentScan`

1. Typically, we do not need to declare `@ComponentScan` explicitly because it is already included inside `@SpringBootApplication`.
2. By default, it scans the package of the `@SpringBootApplication` class and all its subpackages.

Package Structure Example:

```
com.example
├── Application.java (Annotated with @SpringBootApplication)
├── service/
├── repository/
└── controller/
```

Arguments of `@ComponentScan`

① `basePackages`

Specifies string-based package paths to scan.

```
@Configuration
@ComponentScan(basePackages = "com.example.service")
public class AppConfig {
}
```

Spring scans the specified package and all its subpackages.

Scanning Multiple Packages:

```
@ComponentScan(
    basePackages = {
        "com.example.service",
        "com.example.repository"
    }
)
```

② `basePackageClasses`

Instead of using hardcoded package name strings, you specify classes. Spring scans the packages containing those classes.

```
@ComponentScan(basePackageClasses = StudentService.class)
```

- **Why use it?** It is type-safe. If you rename or move the package, the compiler catches errors immediately, unlike hardcoded package name strings.

③ `includeFilters`

Allows you to include additional classes that match specific criteria, even if they do not have stereotype annotations.

```
@ComponentScan(  
    includeFilters = @ComponentScan.Filter(  
        type = FilterType.ANNOTATION,  
        classes = MyAnnotation.class  
    )  
)
```

Spring will scan and register classes annotated with `@MyAnnotation`.

④ `excludeFilters`

Used to exclude specific classes from component scanning.

```
@ComponentScan(  
    excludeFilters = @ComponentScan.Filter(  
        type = FilterType.ASSIGNABLE_TYPE,  
        classes = StudentService.class  
    )  
)
```

Even if `StudentService` is annotated with `@Service`, it will not be registered as a Spring bean.

Excluding Packages using Regex:

```
@ComponentScan(  
    basePackages = "com.example",  
    excludeFilters = @ComponentScan.Filter(  
        type = FilterType.REGEX,  
        pattern = "com\\.example\\.\\.service\\.\\.\\.*"   
    )  
)
```

`@ComponentScan.Filter`

Controls custom filter behaviors during component scanning.

Default Scanned Stereotypes:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`
- `@RestController`

Custom Filters:

```
@ComponentScan.Filter(  
    type = FilterType.REGEX,  
    pattern = ".*Controller"  
)
```

Attributes of `@ComponentScan.Filter`:

1. `type`: The type of filter strategy (e.g., `FilterType.ANNOTATION`).
2. `classes`: The target class or annotation type.
3. `pattern`: Used with `FilterType.REGEX` or `FilterType.ASPECTJ`.

```
@ComponentScan.Filter(  
    type = FilterType.REGEX,  
    pattern = ".*Controller"  
)
```

Internal Enum of `FilterType`:

```
public enum FilterType {  
    ANNOTATION,  
    ASSIGNABLE_TYPE,  
    ASPECTJ,  
    REGEX,  
    CUSTOM  
}
```

① `FilterType.ANNOTATION`

Filters based on annotations present on classes.

```
@ComponentScan.Filter(  
    type = FilterType.ANNOTATION,  
    classes = Service.class
```

```
)
```

② ***FilterType.ASSIGNABLE_TYPE***

Filters based on type hierarchy (interfaces or base classes).

- Checks if a class is of the same type, or extends/implements the specified type.

Example:

```
public interface Payment { }

@Service
public class CreditCardPayment implements Payment { }
```

Filter:

```
@ComponentScan.Filter(
    type = FilterType.ASSIGNABLE_TYPE,
    classes = Payment.class
)
```

③ ***FilterType.REGEX***

Filters using regular expressions matching package or class names.

- Example: `pattern = ".*Controller"`

④ ***FilterType.ASPECTJ***

Filters components using AspectJ pointcut expressions.

```
@ComponentScan(
    excludeFilters = @ComponentScan.Filter(
        type = FilterType.ASPECTJ,
        pattern = "com.example.service..*"
    )
)
```

⑤ ***FilterType.CUSTOM***

Allows custom filtering logic by implementing the `TypeFilter` interface.

Step 1: Create a Custom Filter Class

```
public class MyCustomFilter implements TypeFilter {
    @Override
    public boolean match(
```



```

        MetadataReader metadataReader,
        MetadataReaderFactory metadataReaderFactory
    ) throws IOException {
        // Return true to include as bean, false to ignore
        return true;
    }
}

```

Method Explanation:

Spring passes every scanned class to the `match()` method one by one:

- `StudentService.class`
- `EmployeeService.class`
- `AdminService.class`

If you return `true`, Spring registers the class as a bean. If you return `false`, Spring ignores it.

Step 2: Apply the Custom Filter

```

@ComponentScan(
    basePackages = "com.example",
    includeFilters = @ComponentScan.Filter(
        type = FilterType.CUSTOM,
        classes = MyCustomFilter.class
    ),
    useDefaultFilters = false
)

```

Example: Filtering Admin Services Only

```

public class MyCustomFilter implements TypeFilter {
    @Override
    public boolean match(
        MetadataReader metadataReader,
        MetadataReaderFactory metadataReaderFactory
    ) throws IOException {
        String className = metadataReader
            .getClassMetadata()
            .getClassName();
        return className.endsWith("AdminService");
    }
}

```

- **Result:** Only classes ending with `AdminService` will be registered as beans.

What is a Bean?

A Bean is an object that is created, managed, configured, and **destroyed** by the Spring IoC Container.

when does an object become a Bean?

Suppose we write

```
@Component
public class Student {

}
```

When Spring starts, it scans for classes annotated with @Component (or its stereotypes), creates their objects, and stores them as beans in the application context.

Difference

Normal Java

```
Student student = new Student();
```

As you observe here the object is created by the developer not the spring. This how we do it in the normal java programming.

Spring

```
@Component
public class Student {

}
```

As you observe here the object is created and handled by the Spring IOC container.

A Simple Rule to Remember

If you create the object, it is just the java object. If the object is created by the spring then it is called as the bean.

One Important Clarification

You might ask:

```
Does every bean need @Component?
```

The answer is No.

```
@Component is one way to tell Spring to create a bean.
```

Spring can also create beans using:

```
@Bean methods inside a @Configuration class.  
Stereotype annotations like @Service, @Repository, and @Controller (which are  
specialized forms of @Component).
```

Step 1: Bean Instantiation

Bean Instantiation is the first step in the Spring Bean Life Cycle.

Instantiation means:

The process of creating an object from a class.

In Spring, Bean Instantiation refers to the process where the Spring IoC Container creates the bean object.

What is Instantiation?

Consider the following class:

```
public class Student {  
  
}
```

This is only a class (blueprint). No object exists yet.

When we create an object:

```
Student student = new Student();
```

An object is created in memory.

This process is called Instantiation.

Bean Instantiation in Normal Java

```
public class Student {  
  
    public Student() {  
        System.out.println("Student Object Created");  
    }  
  
}  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student student = new Student();  
  
    }  
  
}
```

Output

Student Object Created

Explanation

- The developer creates the object.
- The constructor is executed.
- A Java object is created.
- This is called Java Object Instantiation.

Bean Instantiation in Spring

```
@Component
public class Student {

    public Student() {
        System.out.println("Student Object Created");
    }

}

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {

        SpringApplication.run(DemoApplication.class, args);

    }

}
```

Output

Student Object Created

Explanation

Notice that we never write:

```
new Student();
```

Still, the constructor executes because the Spring IoC Container creates the object internally.

Conceptually, Spring performs something similar to:

```
Student student = new Student();
```

This happens internally and is not written by the developer.

How does Spring know it should create the object?

When the application starts, Spring performs Component Scanning.

It scans the project for classes annotated with annotations such as:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`

At this stage:

- Object is created.
- Constructor is executed.
- Bean exists in memory.

What does NOT happen during Bean Instantiation?

The following lifecycle steps have not occurred yet:

```
Dependency Injection (@Autowired)
```

Step: 2 Dependency Injection

After creating the bean object (Instantiation), Spring injects all the required dependencies (other beans, values, or resources) into the bean before it is initialized.

Step: 3 BeanNameAware

BeanNameAware is an interface in Spring that allows a bean to know its own name inside the Spring IoC Container.

It is part of the **Aware Interfaces** phase in the **Spring Bean Life Cycle**.

In simple words:

BeanNameAware allows Spring to provide the registered bean name to the bean after dependency injection.

BeanNameAware Interface

Package:

`org.springframework.beans.factory.BeanNameAware`

Interface:

```
public interface BeanNameAware {  
  
    void setBeanName(String name);  
  
}
```

It contains only one method:

```
setBeanName(String name)
```

Why BeanNameAware is Required?

When Spring creates a bean, it registers the object inside the IoC Container with a unique name.

Example:

```
@Component
public class Student {

}
```

Spring registers it as: student

The Spring Container knows:

- Bean Name = student
- Object = Student instance

But the bean object itself does not know:

"My name inside Spring is student"

BeanNameAware allows Spring to inform the bean about its registered name.

Why Does a Bean Need to Know Its Name?

- In normal Spring Boot applications, a bean usually does not need to know its own name.
- **BeanNameAware** exists because some beans, especially framework-level or infrastructure components, may need to identify themselves inside the Spring IoC Container.

Use Case 1: Logging and Debugging

A generic component may need to print the name of the bean it is working with.

Example:

- paymentService started
- orderService started
- emailService started

Instead of hardcoding names, the bean can use the name provided by Spring.

Use Case 2: Framework-Level Components

Spring allows developers to create:

- Custom framework components
- Monitoring tools
- Infrastructure components
- Libraries integrated with Spring

Such components may need to know:

Which bean am I working with?

- The bean name helps identify the object inside the container.

Use Case 3: Multiple Beans of Same Class

Example:

```
@Component("indiaPayment")
class PaymentService {

}

@Component("usaPayment")
class PaymentService {

}
```

Spring Container:

- indiaPayment ---> PaymentService Object
- usaPayment ----> PaymentService Object

The bean name helps **distinguish** between **multiple objects** of the **same class**.

Important Point

- ✓ **BeanNameAware** does not mean every bean should know its name.

Most application-level beans such as:

1. Services
2. Controllers
3. Repositories

do not require their bean name.

It is mainly useful for special scenarios where a bean needs information about its identity inside the Spring Container.

Internal Working

Example:

```
@Component
public class Student implements BeanNameAware {

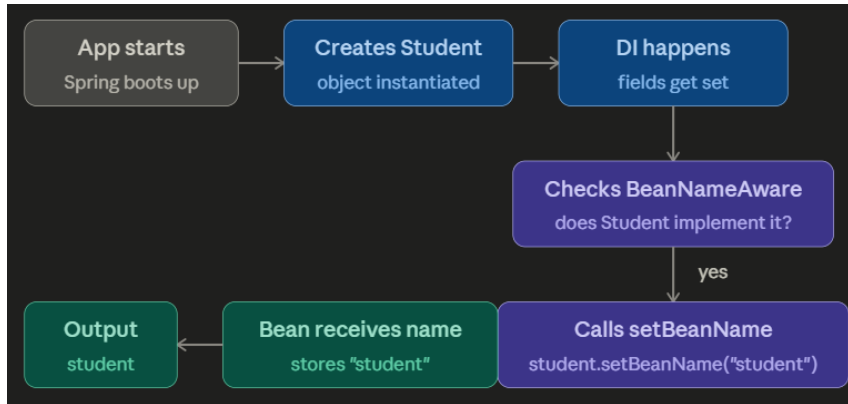
    @Override
    public void setBeanName(String name) {

        System.out.println(name);

    }

}
```

Execution flow:



Default Bean Name

If we do not provide any bean name:

```
@Component
public class Student {

}
```

Spring creates a default bean name.

Rule:

- ✓ Class name with the first letter converted to lowercase.

Examples:

Default Bean Naming Examples

Class Name	Default Bean Name
Student	student
StudentDetails	studentDetails
PaymentService	paymentService
OnlinePaymentService	onlinePaymentService

Custom Bean Name

We can provide our own bean name.

Example:

```
@Component("studentBean")
public class Student {
```



```
}
```

Now Spring registers the bean as: studentBean

When BeanNameAware is called:

```
setBeanName("studentBean")
```

Output: studentBean

BeanNameAware Example with Custom Name

```
@Component("myStudent")
public class Student implements BeanNameAware {

    @Override
    public void setBeanName(String name) {

        System.out.println("Bean Name : " + name);

    }

}
```

Output:

Bean Name : myStudent

Important Points

1. Who calls setBeanName()?

- ✓ The developer does not call it.
- ✓ The Spring IoC Container calls it automatically.

2. When is it called?

- ✓ Bean Instantiation → Dependency Injection → setBeanName() → Bean Initialization

3. Is BeanNameAware mandatory?

No.

It is an optional lifecycle callback.

Only beans implementing this interface receive this callback.

Bean Name vs Class Name

They are different.

Example:

```
@Component("studentObject")
public class Student {
```

```
}
```

Class Name: Student

Bean Name: studentObject

Stage: 4 – BeanFactoryAware and ApplicationContextAware (Spring Bean Life Cycle)

BeanFactoryAware is an interface in Spring that allows a bean to get a reference to the BeanFactory that manages the bean.

In simple words:

- ✓ BeanFactoryAware makes a bean aware of the Spring BeanFactory container.
- ✓ It is part of the Aware Interfaces phase in the Spring Bean Life Cycle.

What is BeanFactory?

BeanFactory is the basic IoC container interface provided by Spring.

Its responsibility is:

- ✓ Creating beans.
- ✓ Managing beans.
- ✓ Providing beans from the container.

Why Does a Bean Need BeanFactory?

Normally, a bean does not know anything about the container.

Example:

```
@Component
public class Student {

}
```

The Student object does not know:

- ✓ Who created it?
- ✓ Where other beans are stored?
- ✓ How to access other beans?

BeanFactoryAware provides a reference to the BeanFactory.

BeanFactoryAware Interface

Package:

org.springframework.beans.factory.BeanFactoryAware

Interface:

```
public interface BeanFactoryAware {
```

```
void setBeanFactory(BeanFactory beanFactory) throws BeansException;

}
```

Method:

```
setBeanFactory()
```

Internal Working

Example:

```
@Component
public class Student implements BeanFactoryAware {

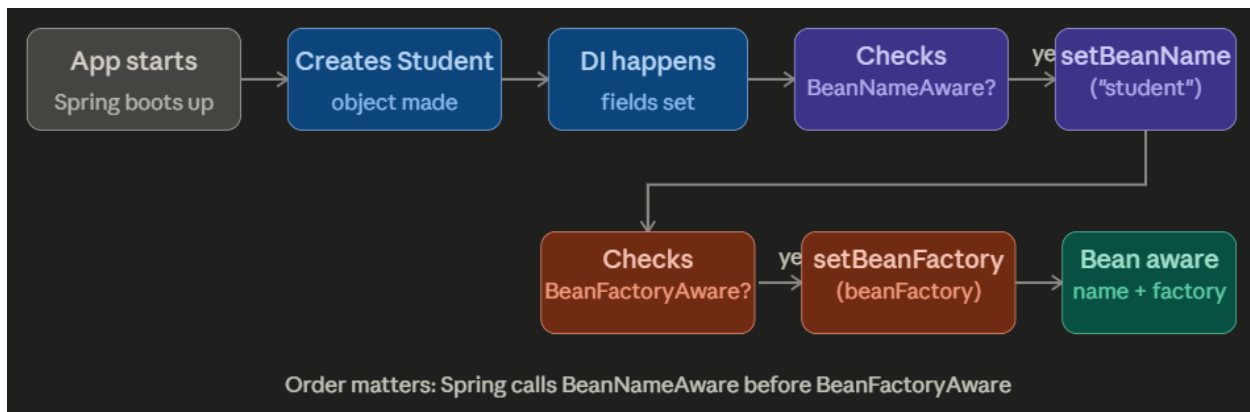
    @Override
    public void setBeanFactory(BeanFactory beanFactory) {

        System.out.println("BeanFactory injected");

    }

}
```

Execution Flow:



- ✓ Bean receives BeanFactory reference

What Can a Bean Do After Receiving BeanFactory?

The bean can access the container.

Example:

```
beanFactory.getBean()
```

It can retrieve other beans from the Spring container.

Important Point

BeanFactoryAware provides only the BeanFactory reference.

It does not provide advanced Spring features like:

- ✓ Event publishing
- ✓ Environment access
- ✓ Internationalization
- ✓ Resource loading

Those features are provided by `ApplicationContext`.

Real-Time Usage

In normal Spring Boot applications, developers rarely use `BeanFactoryAware`.

It is mainly used for:

- ✓ Framework development
- ✓ Custom Spring components
- ✓ Infrastructure-level code
- ✓ Advanced container integration

Step: 5 ApplicationContextAware

`ApplicationContextAware` is an interface in Spring that allows a bean to get a reference to the `ApplicationContext` container.

In simple words:

`ApplicationContextAware` makes a bean aware of the complete Spring application container.

What is ApplicationContext?

`ApplicationContext` is an advanced Spring container.

Relationship:

ApplicationContext extends *BeanFactory*

`ApplicationContext` provides all `BeanFactory` features plus additional features.

It provides:

- ✓ Bean management
- ✓ Event handling
- ✓ Environment access
- ✓ Resource loading
- ✓ Internationalization support

Why Does a Bean Need ApplicationContext?

Normally, a bean does not know about the application container.

Example:

```
@Component
public class Student {

}
```

The bean does not know:

- ✓ What beans exist in the container.
- ✓ Application environment information.
- ✓ Application-level resources.

`ApplicationContextAware` provides access to the complete application container.

ApplicationContextAware Interface

Package:

`org.springframework.context`.`ApplicationContextAware`

Interface:

```
public interface ApplicationContextAware {  
  
    void setApplicationContext(ApplicationContext applicationContext)  
        throws BeansException;  
  
}
```

Method:

```
setApplicationContext()
```

Internal Working

Example:

```
@Component  
public class Student implements ApplicationContextAware {  
  
    @Override  
    public void setApplicationContext(ApplicationContext context) {  
  
        System.out.println("ApplicationContext injected");  
  
    }  
  
}
```

Difference between the ApplicationContextAware and BeanFactoryAware

Aspect	BeanFactoryAware	ApplicationContextAware
Provides	BeanFactory reference	ApplicationContext reference
Container level	Basic container	Advanced container
Access scope	Lightweight container access	Complete application container access
Feature set	Fewer features	More features
Interface level	Lower-level interface	Higher-level interface

Module 6 – BeanPostProcessor (postProcessBeforeInitialization() and postProcessAfterInitialization())

`BeanPostProcessor` is an interface provided by Spring that allows developers or Spring itself to process a bean object during its lifecycle before and after initialization.

In simple words:

`BeanPostProcessor` provides a checkpoint where Spring gives an opportunity to inspect, modify, or wrap a bean before and after its initialization.

Why BeanPostProcessor is Required?

During the Bean Life Cycle, Spring performs many steps:

- ✓ Bean Creation
- ✓ Dependency Injection
- ✓ Initialization
- ✓ Bean Ready

Spring needed a place where it can perform additional processing on beans.

That place is:

`BeanPostProcessor`

It works on the Spring-managed bean object, not on the Java class.

What Does BeanPostProcessor Process?

BeanPostProcessor processes:

Spring Bean Object

Example:

Class:

```
public class Student {  
  
}
```

Spring creates:

```
Student student = new Student();
```

BeanPostProcessor works on:

Student Object

not:

Student.java class

BeanPostProcessor Interface

Package:

org.springframework.beans.factory.config.**BeanPostProcessor**

Interface:

```
public interface BeanPostProcessor {

    Object postProcessBeforeInitialization(
        Object bean,
        String beanName
    );

    Object postProcessAfterInitialization(
        Object bean,
        String beanName
    );

}
```

It contains two methods:

1. postProcessBeforeInitialization()

Definition

```
postProcessBeforeInitialization() is called after dependency injection and aware interfaces but before the initialization methods of the bean are executed.
```

In simple words:

Before initializing the bean, Spring gives one chance to process the bean.

Flow of postProcessBeforeInitialization()

- ✓ Bean Created
- ✓ Dependencies Injected
- ✓ Aware Interfaces Executed
- ✓ postProcessBeforeInitialization()
- ✓ Initialization Methods

Method Signature

```
Object postProcessBeforeInitialization(  
    Object bean,  
    String beanName  
);
```

Parameters

1. Object bean

The actual bean object created by Spring.

Example:

Student Object

2. String beanName

The name of the bean inside the Spring Container.

Example:

student

Internal Working

Suppose we have:

```
@Component  
public class Student {  
  
    @PostConstruct  
    public void init(){  
        System.out.println("Initialization");  
    }  
}
```

BeanPostProcessor:

```
@Component  
public class MyProcessor  
implements BeanPostProcessor {  
  
    @Override  
    public Object postProcessBeforeInitialization(  
        Object bean,  
        String beanName){  
  
        System.out.println(  
            "Before Initialization : "  
            + beanName  
        );  
  
        return bean;  
    }  
}
```



```
}
```

Execution Order

Step 1: Bean Instantiation

Spring creates object:

```
Student student = new Student();
```

Step 2: Dependency Injection

Dependencies are injected.

Example:

Student

Step 3: Aware Interfaces

Spring calls:

```
setBeanName()  
setBeanFactory()  
setApplicationContext()
```

if implemented.

Step 4: BeanPostProcessor

Spring calls:

```
postProcessBeforeInitialization(  
    student,  
    "student"  
);
```

Step 5: Initialization

Spring executes:

```
@PostConstruct
```

What Can Be Done Inside Before Method?

1. Checking the Bean
2. Modifying the Bean

Example:

Before:

Student name = null

Processor changes:

Student name = Vineeth

3. Processing Annotations

2. *postProcessAfterInitialization()*

Definition

```
postProcessAfterInitialization() is called after all initialization methods are completed but before the bean is given to the application.
```

In simple words:

After the bean is initialized, Spring gives another chance to process the final bean.

Flow of postProcessAfterInitialization()

Bean Created → Dependency Injection → Before Initialization → Initialization Completed → *postProcessAfterInitialization()*

Why Do We Need After Initialization?

After initialization, Spring may need to:

- ✓ Wrap the bean.
- ✓ Create proxy objects.
- ✓ Add extra behavior.

The most common example:

AOP Proxy Creation

Original object:

PaymentService Object

Spring creates:

- ✓ PaymentService Proxy Object
- ✓ **The proxy adds:**
- ✓ Transactions
- ✓ Security
- ✓ Logging
- ✓ AOP functionality

Difference Between *postProcessBeforeInitialization* and *postProcessAfterInitialization*

<i>postProcessBeforeInitialization()</i>	<i>postProcessAfterInitialization()</i>
Runs before initialization methods	Runs after initialization methods
Bean is not fully initialized	Bean is fully initialized
Used for checking/modifying bean before initialization	Used for final processing or wrapping bean

Happens before @PostConstruct

Happens after @PostConstruct

Simple Example

```
@Component
public class MyProcessor
implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(
        Object bean,
        String beanName){

        System.out.println(
            "Before : " + beanName
        );

        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(
        Object bean,
        String beanName){

        System.out.println(
            "After : " + beanName
        );

        return bean;
    }
}
```

Output:

Before : student

```
@PostConstruct
```

After : student

@PostConstruct (Spring Bean Life Cycle)

Definition

@PostConstruct is an annotation provided by Spring that marks a method to be executed after dependency injection is completed and before the bean is ready for use.

In simple words:

`@PostConstruct` is used to perform initialization logic after Spring creates the bean and injects all required dependencies.

Why Does `@PostConstruct` Exist?

During the Bean Life Cycle:

- ✓ Bean Creation
- ✓ Dependency Injection
- ✓ Initialization
- ✓ Bean Ready

After dependency injection, developers may need to perform some setup operations.

Examples:

- ✓ Loading initial data.
- ✓ Checking configurations.
- ✓ Creating cache.
- ✓ Opening resources.
- ✓ Performing startup logic.

For this purpose Spring provides:

`@PostConstruct`

Why Does `@PostConstruct` Run After Dependency Injection?

Spring follows this order:

1. Create Bean Object
2. Inject Dependencies
3. Execute `@PostConstruct`
4. Bean Ready

Example:

```
@Component
public class StudentService {

    @Autowired
    private StudentRepository repository;

    @PostConstruct
    public void init(){

        repository.loadData();

    }

}
```

When `init()` executes:

StudentService Object

The repository is already injected.

Why Explicitly Mention "After Dependency Injection"?

All initialization methods in Spring happen after dependency injection.

But the reason we specifically mention it for `@PostConstruct` is:

It tells developers that inside this method they can safely use injected dependencies.

Example:

Before Dependency Injection:

repository = null

After Dependency Injection:

repository = StudentRepository Object

Now:

```
@PostConstruct
public void init(){

    repository.findAll();

}
```

is safe.

Constructor vs @PostConstruct

Constructor

Constructor runs during object creation.

Flow:

- ✓ Bean Instantiation
- ✓ Constructor
- ✓ Dependency Injection

```
@PostConstruct
```

Example:

```
@Component
public class StudentService {

    public StudentService(){

        // Object is created here

    }

}
```

```
}
```

At constructor time:

Object exists.

Dependencies may not be injected yet.

```
@PostConstruct
```

Runs after Spring completes object preparation.

Example:

```
@Component
public class StudentService {

    @Autowired
    private StudentRepository repository;

    @PostConstruct
    public void init(){

        repository.findAll();

    }

}
```

At this stage:

Object exists.

Dependencies are injected.

Initialization logic can execute.

Syntax

```
@PostConstruct
public void initialize(){

    // initialization logic

}
```

Rules:

Method should not have parameters.

```
Method should return void.
```

Method is executed only once.

Who Calls @PostConstruct?

The developer does not call it.

You do not write:

```
studentService.initialize();
```

Spring automatically calls it during the Bean Life Cycle.

Internally:

Spring Container

Finds `@PostConstruct` method

Calls the method automatically

Practical Example

```
@Component
public class DatabaseService {

    @Autowired
    private DataSource dataSource;

    @PostConstruct
    public void start(){

        System.out.println(
            "Database Service Initialized"
        );

    }

}
```

Execution:

Create DatabaseService Object

Inject DataSource

Execute start()

Bean Ready

Why Developers Prefer @PostConstruct?

```
@PostConstruct is commonly used because it is:
```

Simple.

Clean.

Easy to understand.

Does not require implementing Spring interfaces.

Example:

```
@PostConstruct
public void init(){

}
```

Compared with:

```
implements InitializingBean
@PostConstruct keeps the class less tightly coupled with Spring.
```

Real-Time Usage Examples

1. Loading Initial Data

```
@PostConstruct
public void loadData(){

    loadDefaultUsers();

}
```

2. Cache Initialization

```
@PostConstruct
public void createCache(){

    initializeCache();

}
```

3. Configuration Validation

```
@PostConstruct
public void validate(){

    checkConfiguration();

}
```

Important Points

@PostConstruct is part of the Bean Initialization phase.

- ✓ It runs after dependency injection.
- ✓ It runs before the bean is available to the application.
- ✓ Spring automatically calls this method.
- ✓ It is used for startup and initialization logic.
- ✓ It executes only once per bean instance.
- ✓ Injected dependencies can safely be used inside this method.

InitializingBean (afterPropertiesSet()) - Spring Bean Life Cycle

Definition

`InitializingBean` is a Spring interface that provides a lifecycle callback method:

```
afterPropertiesSet()
```

This method is executed after Spring completes dependency injection and sets all bean properties.

In simple words:

`InitializingBean` allows a bean to perform initialization logic after all its dependencies are injected by Spring.

Why Do We Need InitializingBean?

After Spring creates a bean and injects all dependencies, sometimes the bean needs to perform some setup operations.

Examples:

- ✓ Validating injected dependencies.
- ✓ Loading initial data.
- ✓ Creating resources.
- ✓ Performing startup logic.

For this purpose Spring provides:

`InitializingBean`

InitializingBean Interface

Package:

org.springframework.beans.factory.`InitializingBean`

Interface:

```
public interface InitializingBean {  
  
    void afterPropertiesSet() throws Exception;  
  
}
```

It contains only one method:

```
afterPropertiesSet()
```

Why the Name afterPropertiesSet()?

Let's understand the meaning.

Properties mean:

- ✓ Bean fields.
- ✓ Injected dependencies.

- ✓ Configuration values.

Example:

```
@Component
public class StudentService {

    private StudentRepository repository;

}
```

Before dependency injection:

repository = null

After dependency injection:

repository = StudentRepository Object

Now all properties are set.

Then Spring calls:

```
afterPropertiesSet()
```

Meaning:

"All required properties are set, now execute initialization logic."

Example

```
@Component
public class StudentService
    implements InitializingBean {

    @Autowired
    private StudentRepository repository;

    @Override
    public void afterPropertiesSet(){

        System.out.println(
            "StudentService Initialized"
        );

    }

}
```

Internal Working

Step 1: Bean Instantiation

Spring creates the object:

```
StudentService service = new StudentService();
```

At this stage:

repository = null

Step 2: Dependency Injection

Spring injects dependencies:

```
service.repository = repositoryObject;
```

Now:

repository = StudentRepository Object

Step 3: Spring Checks `InitializingBean`

Spring checks:

Does this bean implement `InitializingBean`?

Example:

```
implements InitializingBean
```

Answer:

Yes

```
Step 4: Spring Calls afterPropertiesSet()
```

Spring internally calls:

```
service.afterPropertiesSet();
```

Step 5: Bean Ready

After initialization, the bean becomes available for use.

Complete Execution Order

Create Bean Object

Inject Dependencies

```
@PostConstruct executes  
afterPropertiesSet() executes  
Custom initMethod executes
```

Bean Ready

Difference Between `@PostConstruct` and `InitializingBean`

<code>@PostConstruct</code>	<code>InitializingBean</code>
Annotation based	Interface based

Uses <code>@PostConstruct</code> annotation	Uses <code>afterPropertiesSet()</code> method
Java standard approach	Spring-specific approach
Less coupling with Spring	Tightly coupled with Spring
Preferred in most applications	Mostly used in framework-level code

Mostly used in framework-level code

Why Is @PostConstruct Preferred?

Using InitializingBean:

```
public class StudentService
    implements InitializingBean {

}
```

The class directly depends on Spring.

Using @PostConstruct:

```
@PostConstruct
public void init(){

}
```

The class remains cleaner.

Important Points

`InitializingBean` is a Spring lifecycle interface.

It contains only one method:

```
afterPropertiesSet()
```

Spring automatically calls this method.

Developers do not call it manually.

It executes after dependency injection.

It executes after `@PostConstruct`.

```
It executes before custom initMethod.
```

It is used for bean initialization logic.

Key Takeaway

```
InitializingBean provides the afterPropertiesSet() callback method, which  
Spring executes after setting all bean properties and dependencies, allowing  
the bean to perform initialization tasks before becoming ready for use.
```

Custom Init Method (initMethod) - Spring Bean Life Cycle

Definition

A Custom Init Method is a user-defined method that Spring calls automatically during the bean initialization phase.

In simple words:

Custom Init Method allows developers to create their own initialization method and configure Spring to execute it after dependency injection is completed.

Why Do We Need Custom Init Method?

Spring provides multiple ways to perform initialization:

```
@PostConstruct  
InitializingBean (afterPropertiesSet())
```

Custom Init Method (initMethod)

The problem with InitializingBean:

```
implements InitializingBean
```

The class becomes tightly coupled with Spring.

The advantage of Custom Init Method:

The class remains a normal Java class.

No Spring annotations are required inside the class.

Initialization logic can be controlled from configuration.

How Custom Init Method Works?

The process:

1. Create a normal Java class
2. Write a custom initialization method
3. Configure Spring with initMethod name
4. Spring automatically calls that method

Example

Bean Class

```
public class Student {  
  
    public void initialize(){  
  
        System.out.println(  
            "Student initialized"  
        );  
  
    }  
  
}
```

Notice:

The class does not contain:

```
@PostConstruct
```

or

```
implements InitializingBean
```

It is a simple Java class.

Spring Configuration

```
@Configuration  
public class AppConfig {  
  
    @Bean(initMethod = "initialize")  
    public Student student(){  
  
        return new Student();  
  
    }  
  
}
```

Here:

```
@Bean(initMethod = "initialize")
```

tells Spring:

"After creating this bean, call the initialize() method."

Internal Working

Step 1: Bean Instantiation

Spring creates the object:

```
Student student = new Student();
```

Step 2: Dependency Injection

Spring injects required dependencies.

Example:

Student

```
Step 3: Spring Finds initMethod
```

Spring reads:

```
@Bean(initMethod = "initialize")
```

It knows:

Call initialize()

Step 4: Spring Calls Custom Method

Internally:

```
student.initialize();
```

Output:

Student initialized

Step 5: Bean Ready

After initialization:

Bean Ready for Use

Custom Init Method Execution Order

If all initialization approaches are present:

```
BeanPostProcessor
```

```
(postProcessBeforeInitialization)
```

```
@PostConstruct  
afterPropertiesSet()  
Custom initMethod()
```

```
BeanPostProcessor
```

```
(postProcessAfterInitialization)
```

Method Name in Custom Init Method

The method name can be anything.

Example:

```
@Bean(initMethod = "start")
```

Bean:

```
public void start(){  
  
}
```

or:

```
@Bean(initMethod = "connectDatabase")
```

Bean:

```
public void connectDatabase(){  
  
}
```

Spring only checks whether the configured method exists.

Rules for Custom Init Method

1. No Parameters

Correct:

```
public void init(){  
  
}
```

Wrong:

```
public void init(String name){  
  
}
```

2. Return Type Should Be void

Correct:

```
public void init(){  
  
}
```

Wrong:

```
public String init(){  
  
}
```

Example with Dependency

Bean:

```
public class DatabaseService {  
  
    private DataSource dataSource;
```



```

    public DatabaseService(DataSource dataSource){

        this.dataSource = dataSource;

    }

    public void connect(){

        System.out.println(
            "Database Connected"
        );

    }

}

```

Configuration:

```

@Bean(initMethod = "connect")
public DatabaseService databaseService(){

    return new DatabaseService(dataSource());

}

```

Flow:

Create DatabaseService

Inject DataSource

```

Call connect()

```

Bean Ready

Difference Between Bean Initialization Methods

@PostConstruct	InitializingBean	Custom Init Method
Annotation based	Interface based	Configuration based
Uses @PostConstruct	Uses afterPropertiesSet()	Uses custom method name
Less coupling	Tightly coupled with Spring	No Spring dependency inside class
Most commonly used	Mostly framework-level	Useful for POJO and third-party classes

When To Use Custom Init Method?

Use Custom Init Method when:

- ✓ You want a clean POJO class.
- ✓ You don't want Spring-specific code inside your class.
- ✓ You are working with third-party classes.
- ✓ Initialization should be controlled through configuration.

Example:

Third-party class:

```
public class PaymentClient {  
  
    public void start(){  
  
    }  
  
}
```

Cannot modify the class.

Spring configuration:

```
@Bean(initMethod = "start")
```

Real-Time Example

Factory machine:

- ✓ Machine Created
- ✓ Parts Installed
- ✓ Startup Procedure
- ✓ Machine Ready

The startup procedure represents:

Custom Init Method

Important Points

Custom Init Method is part of the Bean Initialization phase.

It executes after dependency injection.

Spring calls it automatically.

Developers do not call it manually.

It is configured using:

```
@Bean(initMethod = "methodName")
```

The method should have:

No parameters.

```
void return type.
```

It keeps the Java class independent from Spring.

Key Takeaway

Custom Init Method is a user-defined initialization method configured in Spring that executes after dependency injection and before the bean becomes ready for use.

Application Running, @PreDestroy and DisposableBean (destroy()) - Spring Bean Life Cycle

Application Running Phase

After all initialization steps are completed, the Spring Bean becomes ready for use.

What Happens During Application Running?

During this phase:

- ✓ Spring Container manages all beans.
- ✓ Application uses these beans.
- ✓ Beans perform their actual business operations.

Example:

Request

Controller Bean

Service Bean

Repository Bean

The bean remains active until the Spring Container is closed.

Why Do We Need Destruction Methods?

Many beans use external resources:

Examples:

1. Database connections.
2. File connections.
3. Network connections.
4. Thread pools.
5. Cache resources.
6. Sockets.

When the application stops, these resources should be released properly.

Example:

1. Application Shutdown
2. Release Resources
3. Destroy Bean

For this Spring provides destruction callbacks.

Why Do We Use @PreDestroy?

It is used for cleanup tasks:

Examples:

1. Closing database connections.
2. Closing files.
3. Stopping background threads.
4. Releasing resources.

Syntax

@PreDestroy

```
public void cleanup(){  
  
    // cleanup logic  
  
}
```

Example

```
@Component  
public class DatabaseService {
```

@PreDestroy

```
    public void cleanup(){  
  
        System.out.println(  
            "Closing database connection"  
        );  
  
    }  
  
}
```

Internal Working of @PreDestroy

Step 1: Application Starts

Spring creates the bean:

DatabaseService Object

Step 2: Initialization

Bean goes through initialization:

```
@PostConstruct  
afterPropertiesSet()  
initMethod()
```

Step 3: Application Running

Bean performs its work.

Step 4: Application Shutdown

Spring Container starts closing.

Step 5: Spring Calls `@PreDestroy`

Spring automatically executes:

```
cleanup();
```

Step 6: Bean Destroyed

Bean is removed from the container.

Real-Time Example

```
@Component
public class DatabaseConnection {

    @PostConstruct
    public void connect(){

        System.out.println(
            "Database Connected"
        );

    }
}
```

@PreDestroy

```
    public void disconnect(){

        System.out.println(
            "Database Disconnected"
        );

    }

}
```

Execution:

Application Start:

Database Connected

Application Shutdown:

Database Disconnected

2. DisposableBean (destroy())

Definition

`DisposableBean` is a Spring interface that provides a destruction callback method:

```
destroy()
```

Spring calls this method before **destroying** the bean.

Interface

```
public interface DisposableBean {  
  
    void destroy() throws Exception;  
  
}
```

Example

```
@Component  
public class DatabaseService  
    implements DisposableBean {  
  
    @Override  
    public void destroy() {  
  
        System.out.println(  
            "Cleaning resources"  
        );  
  
    }  
  
}
```

Internal Working of DisposableBean

Spring checks:

Does this bean implement **DisposableBean**?

If yes:

Call **destroy()**

Flow:

Application Shutdown

Spring Container Closing

```
destroy()
```

Bean Removed

Difference Between @PreDestroy and DisposableBean

@PreDestroy	DisposableBean
Annotation based	Interface based

Uses @PreDestroy annotation	Uses destroy() method
Less coupling with Spring	Tightly coupled with Spring
Preferred in modern Spring Boot	Older Spring approach
Cleaner approach	Requires implementing Spring interface

Requires implementing Spring interface

Execution Order During Destruction

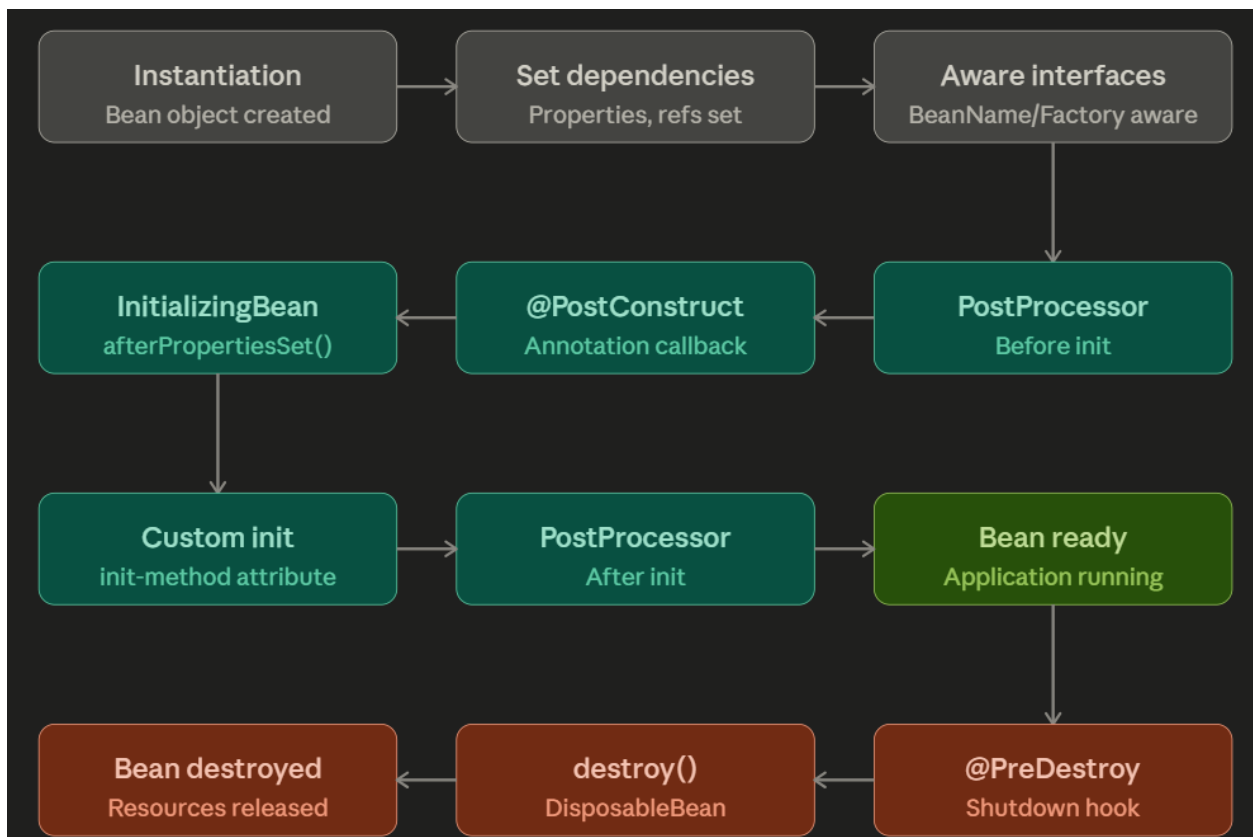
If both are present:

Application Shutdown

@PreDestroy

```
DisposableBean.destroy()
```

Bean Destroyed



AOP (Aspect oriented Programming)

1. Aspect

An Aspect is a class that contains cross-cutting concerns (such as logging, security, transactions, etc.) and defines where and when those concerns should be applied.

What does the Cross-Cutting-concerns mean?

Cross-Cutting Concern: A functionality that is needed by multiple modules or classes throughout an application, regardless of their primary business purpose.

Examples:

- Logging
- Security
- Transactions
- Auditing
- Exception Handling
- Performance Monitoring

Why do we need an Aspect?

Suppose we have three services:

```
UserService  
PaymentService  
OrderService
```

All of them need:

- Logging
- Security
- Transactions

Without AOP:

UserService -> Logging -> Security

PaymentService -> Logging -> Security

OrderService -> Logging -> Security

We are repeating the same code everywhere.

Spring says:

"Move all common functionality into one class."

That class is called an **Aspect**.

An **Aspect** mainly contains these components:

1. **Advice**
2. **Pointcut**

These are the two primary components of an Aspect.

Additionally, while working with Spring AOP, we also deal with:

3. **Join Point**
4. **Pointcut Expression**

Short Notes:

1. **Advice** → Defines **what action to perform**
Example: Logging, Security check, Transaction handling
2. **Pointcut** → Defines **where the Advice should be applied**
Example: Apply logging to all service methods
3. **Join Point** → Defines **the execution point where an Aspect can be applied**
Example: Method execution
4. **Pointcut Expression** → Defines **the rule/pattern to select Join Points**
Example: `execution(* com.example.service.*.*(..))`

① Advice

Advice is the actual code or action that gets executed when a specific Join Point is reached.

In simple words:

Advice tells Spring AOP what work should be performed when a matched method executes.

Why do we need Advice?

An Aspect only represents a cross-cutting concern.

Example:

- LoggingAspect
- SecurityAspect
- TransactionAspect

But Spring needs to know:

"What should I actually do?"

That actual work is written inside **Advice**.

Example:

```
public void logMessage() {  
    System.out.println("Method execution started");  
}
```

This logging code is an Advice.

Suppose we have a service:

```
@Service  
public class PaymentService {  
    public void makePayment() {  
        System.out.println("Payment processing...");  
    }  
}
```

We want logging before this method.

Without AOP:

```
public void makePayment() {  
    System.out.println("Method started");  
    // Payment logic  
}
```

The logging code is mixed with business logic.

With AOP:

```
@Aspect  
@Component  
public class LoggingAspect {  
    @Before("execution(* com.example.service.*.*(..))")  
    public void log() {  
        System.out.println("Method started");  
    }  
}
```

Here:

log()

is the **Advice**.

Spring automatically executes it before makePayment().

Types of Advice

Spring AOP provides 5 types of Advice:

1. @Before Advice

@Before Advice is an advice that executes before the execution of the target method.

Example:

```
@Before("execution(* com.example.service.*.*(..))")
public void logBefore() {
    System.out.println("Method execution started");
}
```

2. @After Advice

@After Advice executes after the target method execution, regardless of whether the method completes successfully or throws an exception.

Example:

```
@After("execution(* com.example.service.*.*(..))")
public void logAfter() {
    System.out.println("Method execution completed");
}
```

3. @AfterReturning Advice

@AfterReturning Advice executes after the target method successfully completes and returns a result.

Example:

```
@AfterReturning(
    pointcut = "execution(* com.example.service.*.*(..))",
    returning = "result"
)
public void logResult(Object result) {
    System.out.println("Result: " + result);
}
```

4. @AfterThrowing Advice

@AfterThrowing Advice executes when the target method throws an exception.

Example:

```
@AfterThrowing(
    pointcut = "execution(* com.example.service.*.*(..))",
    throwing = "exception"
)
```

```

)

public void logException(Exception exception) {
    System.out.println("Error: " + exception.getMessage());
}

```

5. @Around Advice

@Around Advice surrounds the target method execution and can execute logic before and after the method, and can control whether the method executes.

Example:

```

@Around("execution(* com.example.service.*.*(..))")
public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {

    System.out.println("Before method");

    Object result = joinPoint.proceed();

    System.out.println("After method");

    return result;
}

```

2. PointCut

A Pointcut is an expression that identifies the Join Points where an Advice should be applied.

In simple words:

Pointcut tells Spring AOP "where to apply the Advice".

Example

Suppose we have multiple methods:

- ❖ UserService
 - saveUser()
- ❖ PaymentService
 - makePayment()
- ❖ OrderService
 - CreateOrder()

We have a Logging Advice:

```
@Before(...)  
  
public void log(){  
    System.out.println("Logging");  
}
```

Now the question is:

Where should this logging execute?

- Only UserService?
- All services?
- Only save methods?

The answer is decided by the **Pointcut**.

Example:

```
execution(* com.example.service.*.*(..))
```

Meaning:

Apply this Advice to all methods inside the service package.

Relationship between Advice and Pointcut

```
Aspect  
|  
|---- Advice  
|  |  
|  |---- What should happen?  
|  
|---- Pointcut  
|  
|---- Where should it happen?
```

Example:

```
@Before("execution(* com.example.service.*.*(..))")  
  
public void log(){  
    System.out.println("Logging");  
}
```

Here:

- @Before → Advice
- execution(...) → Pointcut expression

@Pointcut Annotation

Spring provides @Pointcut to create reusable pointcuts.

Example:

```
@Pointcut("execution(* com.example.service.*.*(..))")
public void serviceMethods() {
}
```

Now we can reuse:

```
@Before("serviceMethods()")
public void log() {
    System.out.println("Logging");
}
```

Types of Pointcut Expressions

1. execution()

execution() Pointcut expression is used to match the execution of methods based on method signature.

In simple words:

It tells Spring AOP **which methods should be intercepted based on their return type, method name, parameters, and package/class location.**

Syntax

```
execution(modifiers? return-type declaring-class? method-name(parameters))
```

Common format:

```
execution(return-type package.class.methodName(parameters))
```

Example

```
@Before("execution(* com.example.service.UserService.saveUser(..))")
public void log() {
    System.out.println("Logging");
}
```

Meaning:

- * → Any return type
- com.example.service.UserService → Specific class
- saveUser → Specific method
- (..) → Any number of parameters

So this Advice runs only when: **saveUser()** executes.

Example 2: All methods in a package

```
@Before("execution(* com.example.service.*.*(..))")
public void log() {
    System.out.println("Logging");
}
```

Meaning:

- Any return type
- Any class inside service package
- Any method
- Any parameters

Matches:

- ❖ UserService.saveUser()
- ❖ OrderService.createOrder()
- ❖ PaymentService.pay()

Example 3: Specific method pattern

```
execution(* save*(..))
```

Matches:

- ❖ saveUser()
- ❖ saveOrder()
- ❖ saveProduct()

Because method names start with save.

2.within()

within() Pointcut expression is used to match all method executions inside a specified class or package.

It selects methods based on **where the class belongs**, not based on the method name or parameters.

Example

```
@Before("within(com.example.service.UserService)")
public void log() {
    System.out.println("Logging");
}
```

Here, the Advice will be applied to all methods inside:

UserService

Example methods:

```
saveUser()  
updateUser()  
deleteUser()
```

Important Points

Point	Description
Purpose	Selects methods based on class/package location
Matches	All methods inside a class or package
Focus	Class-level matching
Does not check	Method name, return type, parameters
Common Usage	Apply common logic to an entire layer/package
Example Pattern	<code>within(com.example.service.*)</code>

Difference with execution()

execution()	within()
Matches method execution details	Matches class/package location
More specific	More general
Can check method name, return type, parameters	Only checks class/package

3. args()

args() Pointcut expression is used to match method executions based on the runtime type of method arguments (parameters).

In simple words:

It applies Advice to methods that receive specific types of parameters.

Example

```
@Before("args(String)")  
public void validateInput() {  
    System.out.println("String argument method called");  
}
```

Here, the Advice will execute for methods that have a String parameter.

Example:

```
public void saveUser(String name)
```


The method matches because it accepts a String argument.

4. @annotation()

@annotation() Pointcut expression is used to match method executions where the method is annotated with a specific annotation.

In simple words:

It applies Advice only to methods that contain a particular annotation.

Example

Suppose we have a custom annotation:

```
@TrackExecution
public void processPayment() {
    System.out.println("Payment processing");
}
```

Aspect:

```
@Before("@annotation(TrackExecution)")
public void log() {
    System.out.println("Tracking method execution");
}
```

Here, the Advice runs only for methods having:

```
@TrackExecution
```

5. this()

this() Pointcut expression is used to match join points where the proxy object is an instance of a specified type.

In simple words:

It checks the **type of the Spring AOP proxy object** and applies Advice if it matches.

Example

```
@Before("this(com.example.service.UserService)")
public void log() {
    System.out.println("UserService proxy method called");
}
```

Here, the Advice executes when the proxy object created by Spring is of type:

```
UserService
```

6. target()

target() Pointcut expression is used to match join points where the actual target object (the original Spring Bean) is an instance of a specified type.

In simple words:

It checks the **real object behind the proxy** and applies Advice if its type matches.

Example

```
@Before("target(com.example.service.UserService)")
public void log() {
    System.out.println("UserService method called");
}
```

Here, the Advice executes when the actual target object is:

UserService

3. Join Point

Definition

A Join Point is a specific point during the execution of an application where an Aspect can be applied.

In simple words:

A Join Point is a place where Spring AOP can intercept and execute an Advice.

In Spring AOP

Spring supports only **method execution** as a Join Point.

Example:

```
public void saveUser() {
    System.out.println("Saving user");
}
```

The execution of:

saveUser() is a Join Point.

4. Weaving

Weaving is the process of connecting an Aspect (Advice) with the target object at the required Join Point.

In simple words:

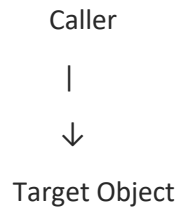
Weaving means adding the Aspect logic to the original business object.

5. Proxy Object in Spring AOP

A Proxy is a Spring-created wrapper object that sits between the caller and the actual target object to intercept method calls and apply additional behavior (Advice).

Why do we need a Proxy Object?

Without a Proxy:

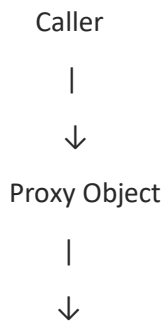


The method call directly reaches the actual object.

Spring has no place to insert:

- Logging
- Security
- Transactions
- Auditing

With a Proxy:



Target Object

The Proxy acts as an intermediate layer.

It can:

1. Execute Advice
2. Call the actual target method

Simple Memory

Proxy = A middle object created by Spring that intercepts calls and applies AOP logic before reaching the actual object.

Types of Proxy in Spring AOP

Spring AOP creates two types of Proxy objects:

1. **JDK Dynamic Proxy**
2. **CGLIB Proxy**

1. JDK Dynamic Proxy

JDK Dynamic Proxy is an interface-based proxy created by Spring when the target class implements an interface.

In simple words:

If a class has an interface, Spring creates a proxy using that interface.

Example

Interface:

```
public interface PaymentService {  
    void pay();  
}
```

Implementation:

```
@Service  
public class PaymentServiceImpl implements PaymentService {  
    public void pay() {  
        System.out.println("Payment done");  
    }  
}
```

Flow:

Caller → JDK Proxy (Interface based) → PaymentServiceImpl

2. CGLIB Proxy

CGLIB Proxy is a class-based proxy created by Spring by creating a subclass of the target class when no interface is available.

In simple words:

If a class does not implement an interface, Spring creates a child class as a proxy.

Example

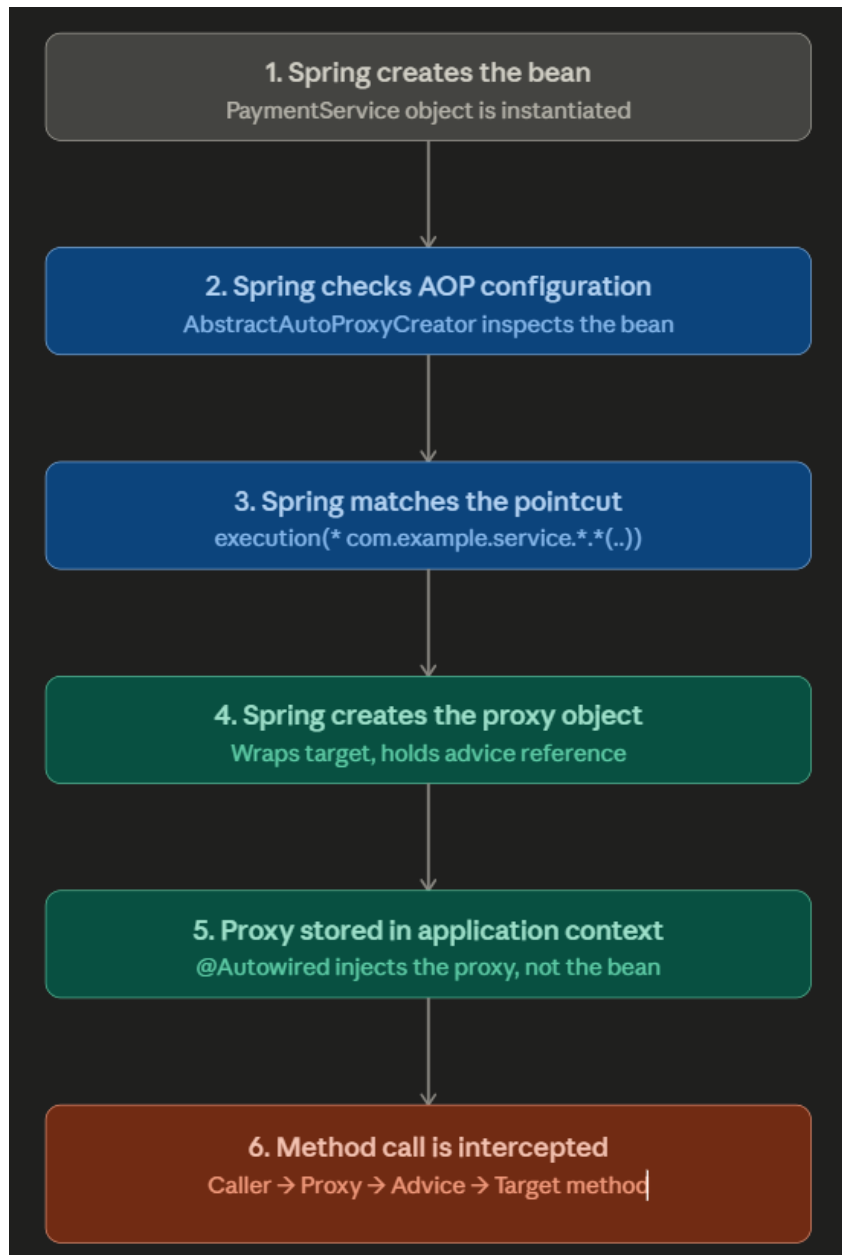
Target Class:

```
@Service  
public class PaymentService {  
    public void pay() {  
        System.out.println("Payment done");  
    }  
}
```

Flow:

Caller → CGLIB Proxy (Subclass) → PaymentService

How does Spring know when to create a Proxy?



Spring uses:

BeanPostProcessor

Specifically:

AbstractAutoProxyCreator

to perform this check.